ECx00G&EC600U&EG800G Series

QuecOpen(SDK) 6-wire SPI NOR Flash File System Mounting API Reference Manual

LTE Standard Module Series

Version: 1.0

Date: 2023-09-14

Status: Released

At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.
Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China
Tel: +86 21 5108 6236
Email: info@quectel.com

Or our local offices. For more information, please visit:
http://www.quectel.com/support/sales.htm.

For technical support, or to report documentation errors, please visit:
http://www.quectel.com/support/technical.htm.
Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

About the Document

Revision History

Version	Date	Author	Description
-	2023-08-03	Sum LI	Creation of the document
1.0	2023-09-14	Sum LI	First official release

Contents

Table Index

Figure Index

1 Introduction

Quectel ECx00G family, EC600U series and EG800G series modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see ***document [1]***.

This document is applicable to QuecOpen® solution based on CSDK build environment. It outlines 6-wire SPI NOR flash file system mounting API, development examples and feature debugging on ECx00G family, EC600U series and EG800G series modules.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC600U Series
ECx00G	EC200G-CN
	EC600G Series
	EC800G-CN
-	EG800G Series

2 hal_spi_flash_prop.csv

hal_spi_flash_prop.csv, the configuration file of 6-wire SPI NOR flash, provided in the CSDK of the module, is in the *components\hal\src* directory. You can add the corresponding flash type in the table according to your specific requirements.

This document outlines the parameter information of the currently supported NOR flash types.

2.1. Supported NOR Flash Types

	mid	capacity	type	wp_type	cpid_type	uid_type
XT25W32B	0x16600b	0x400000	HAL_SPI_FLASH_TYPE_XTX	HAL_SPI_FLASH_WP_GD	0	HAL_SPI_FLASH_UID_SFDP_194H_16
XT25W64B	0x17600b	0x800000	HAL_SPI_FLASH_TYPE_XTX	HAL_SPI_FLASH_WP_GD	0	HAL_SPI_FLASH_UID_SFDP_194H_16
XM25QU64A	0x173820	0x800000	HAL_SPI_FLASH_TYPE_XMCA	HAL_SPI_FLASH_WP_XMCA	0	HAL_SPI_FLASH_UID_SFDP_80H_12
XM25QU64B	0x175020	0x800000	HAL_SPI_FLASH_TYPE_XMCB	0	0	HAL_SPI_FLASH_UID_4BH_16
XM25QU32C	0x165020	0x400000	HAL_SPI_FLASH_TYPE_XMCC	HAL_SPI_FLASH_WP_GD	0	HAL_SPI_FLASH_UID_4BH_8
XM25QU16C	0x155020	0x200000	HAL_SPI_FLASH_TYPE_XMCC	HAL_SPI_FLASH_WP_GD	0	HAL_SPI_FLASH_UID_4BH_8
GD25LE64E	0x1760c8	0x800000	HAL_SPI_FLASH_TYPE_GD	HAL_SPI_FLASH_WP_GD	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8
GD25LQ128C	0x1860c8	0x1000000	HAL_SPI_FLASH_TYPE_GD	HAL_SPI_FLASH_WP_GD	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8
GD25Q127C	0x1840c8	0x1000000	HAL_SPI_FLASH_TYPE_GD	0	0	HAL_SPI_FLASH_UID_4BH_16
W25Q64JV	0x1740ef	0x800000	HAL_SPI_FLASH_TYPE_WINBOND	HAL_SPI_FLASH_WP_GD	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8
P25Q128L	0x186085	0x1000000	HAL_SPI_FLASH_TYPE_PUYA	HAL_SPI_FLASH_WP_GD	0	HAL_SPI_FLASH_UID_4BH_16
P25Q64L	0x176085	0x800000	HAL_SPI_FLASH_TYPE_PUYA	HAL_SPI_FLASH_WP_GD	0	HAL_SPI_FLASH_UID_4BH_16
P25Q32L	0x166085	0x400000	HAL_SPI_FLASH_TYPE_PUYA	HAL_SPI_FLASH_WP_GD	0	HAL_SPI_FLASH_UID_4BH_16
GD	0x0000C8	0x000000	HAL_SPI_FLASH_TYPE_GD	HAL_SPI_FLASH_WP_GD	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8
WINBOND	0x0000EF	0x000000	HAL_SPI_FLASH_TYPE_WINBOND	HAL_SPI_FLASH_WP_GD	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8
XM25QU128	0x184120	0x1000000	HAL_SPI_FLASH_TYPE_XMCC	HAL_SPI_FLASH_WP_XMCA	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8
W25Q128JW	0x1860ef	0x1000000	HAL_SPI_FLASH_TYPE_WINBOND	HAL_SPI_FLASH_WP_GD	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8
W25Q32JW	0x1660ef	0x400000	HAL_SPI_FLASH_TYPE_WINBOND	HAL_SPI_FLASH_WP_GD	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8
GD25LQ32E	0x1660c8	0x400000	HAL_SPI_FLASH_TYPE_GD	HAL_SPI_FLASH_WP_GD	HAL_SPI_FLASH_CPID_4BH	HAL_SPI_FLASH_UID_4BH_8

Figure 1: Parameter Information of Supported Flash Types (1)

	mid	sreg_block_size	sreg_min_num	sreg_max_num	volatile_sr_en	suspend_en	sfdp_en	write_sr12	has_sr2	has_sus1	has_sus2
XT25W32B	0x16600b	1024	0	0	1	0	1	1	1	0	0
XT25W64B	0x17600b	1024	0	0	1	0	1	1	1	0	0
XM25QU64A	0x173820	0	0	0	1	0	1	0	0	0	0
XM25QU64B	0x175020	256	0	3	0	1	1	0	0	0	0
XM25QU32C	0x165020	256	1	3	1	0	1	0	1	1	0
XM25QU16C	0x155020	256	1	3	1	0	1	0	1	1	0
GD25LE64E	0x1760c8	1024	1	3	1	1	1	1	1	1	1
GD25LQ128C	0x1860c8	512	1	3	1	1	1	1	1	1	1
GD25Q127C	0x1840c8	1024	1	3	0	1	1	0	1	1	1
W25Q64JV	0x1740ef	256	1	3	1	1	1	1	1	1	0
P25Q128L	0x186085	1024	1	3	1	0	1	0	1	1	1
P25Q64L	0x176085	1024	1	3	1	1	1	0	1	1	1
P25Q32L	0x166085	1024	1	3	1	1	1	0	1	1	1
GD	0x0000C8	256	1	3	1	1	1	1	1	1	1
WINBOND	0x0000EF	256	1	3	1	1	1	1	1	1	0
XM25QU128	0x184120	1024	1	3	1	1	1	1	1	1	1
W25Q128JW	0x1860ef	256	1	3	1	1	1	1	1	1	0
W25Q32JW	0x1660ef	256	1	3	1	1	1	1	1	1	0
GD25LQ32E	0x1660c8	1024	1	3	1	1	1	1	1	1	1

Figure 2: Parameter Information of Supported Flash Types (2)

NOTE

Not all NOR flash types shown in the figure above have been debugged. Please contact Quectel Technical Support for the information of debugged types.

2.2. Parameter Description

2.2.1. halSpiFlash_t

The parameter information of the NOR flash types in *hal_spi_flash_prop.csv* corresponds to the parameters of *halSpiFlash_t* in *components\hal\include\hal_spi_flash.h*. The structure of flash types:

```
typedef struct halSpiFlash
{
    uintptr_t hwp;
    unsigned mid;
    unsigned capacity;
    uint16_t sreg_block_size;
    uint8_t type : 4;
    uint8_t wp_type : 4;
    uint8_t uid_type : 4;
    uint8_t cpid_type : 4;
    uint8_t sreg_min_num : 4;
    uint8_t sreg_max_num : 4;
    bool volatile_sr_en : 1;
    bool suspend_en : 1;
    bool sfdp_en : 1;
    bool write_sr12 : 1;
    bool has_sr2 : 1;
    bool has_sus1 : 1;
    bool has_sus2 : 1;
} halSpiFlash_t
```

- **Parameter**

Type	Parameter	Description
uintptr_t	*hwp*	6-wire SPI hardware register.
unsigned	*mid*	JEDEC ID consisting of Manufacturer ID and Device ID in little-endian format, which is read with **9Fh** command. For example, JEDEC ID = Manufacturer ID (M7–M0) + Device ID (ID15–ID0), in little-endian format: (ID7–ID0) + (ID15–ID8) + (M7–M0). See ***Chapter 2.2.1.1*** for details.

unsigned	*capacity*	Size of capacity. Unit: byte. Such as: XM25QU32C 2 Mbit (i.e., 0x400000) XM25QU64B 64 Mbit (i.e., 0x800000) GD25LQ128C 128 Mbit (i.e., 0x1000000)
uint16_t	*sreg_block_size*	Size of the security register set. The values can be 256, 512, 1024 and 2048. Unit: byte. The size does not exceed 0xffff bytes. *0* indicates it is not supported. See ***Chapter 2.2.1.2*** for details.
uint8_t	*type*	Manufacturer type of NOR flash. See ***Chapter 2.2.1.3*** for details.
uint8_t	*wp_type*	Protection type. See ***Chapter 2.2.1.7*** for details.
uint8_t	*uid_type*	UID type. See ***Chapter 2.2.1.4*** for details.
uint8_t	*cpid_type*	CPID type. See ***Chapter 2.2.1.5*** for details.
uint8_t	*sreg_min_num*	Minimum security register number. 0 indicates it is not supported.
uint8_t	*sreg_max_num*	Maximum security register number. 0 indicates it is not supported.
bool	*volatile_sr_en*	Whether to support the volatile status register or not. *0* No *1* Yes
bool	*suspend_en*	Whether to support suspend and resume of read-write operation or not. *0* No *1* Yes
bool	*sfdp_en*	Whether to support reading SFDP with **5AH** or not. *0* No *1* Yes
bool	*write_sr12*	Whether to support writing data to SR1 and SR2 status registers through **01H** or not. *0* No *1* Yes
bool	*has_sr2*	Whether to support reading the data from SR2 through **35H** or not. *0* No *1* Yes
bool	*has_sus1*	Whether SUS1 exists in S15 of the status register or not. *0* No *1* Yes
bool	*has_sus2*	Whether SUS2 exists in S10 of the status register or not. *0* No *1* Yes

2.2.1.1. mid

mid = Manufacturer ID followed by Device ID in little-endian format. Compare the *mid* values read from NOR flash with the *mid* values in the table one by one. If the corresponding values are the same, all the information in the table can be read. If not, it indicates that the NOR flash type is not supported. See the examples below for details.

Example 1: The definition in GD25LQ128D Datasheet in the figure below indicates that the value of *mid* is 0x1860c8 (little-endian).

Table of ID Definitions:
GD25LQ128D

Operation Code	M7-M0	ID15-ID8	ID7-ID0
9FH	C8	60	18
90H	C8		17
ABH			17

Figure 3: GD25LQ128D mid

Example 2: From the definition in W25Q128JW Datasheet in the figure below, it can be seen that the value of *mid* is 0x1860ef (little-endian).

8.1.1 **Manufacturer and Device Identification**

MANUFACTURER ID	**(MF7 - MF0)**	
Winbond Serial Flash	EFh	
Device ID	**(ID7 - ID0)**	**(ID15 - ID0)**
Instruction	**ABh, 90h, 92h, 94h**	**9Fh**
W25Q128JW-IQ/JQ	17h	6018h
W25Q128JW-IM/JM*	17h	8018h

Figure 4: W25Q128JW mid

2.2.1.2. Security Register

Whether NOR flash type supports security register is confirmed by the Datasheet of the type. See the following examples for the methods.

Example 1: GD25LQ128D Datasheet information is given in the figure below.

The GD25LQ128D provides three 1024-byte Security Registers which can be erased and programmed individually. These registers may be used by the system manufacturers to store security and other important information separately from the main memory array.

Address	A23-16	A15-12	A11-10	A9-0
Security Register #1	00H	0 0 0 1	0 0	Don't Care
Security Register #2	00H	0 0 1 0	0 0	Don't Care
Security Register #3	00H	0 0 1 1	0 0	Don't Care

Figure 5: GD25LQ128D Datasheet Information

According to the Datasheet:
sreg_block_size: 1024. Size of the security register set. Unit: byte.
sreg_min_num: 1. Minimum security register number.
sreg_max_num: 3. Maximum security register number.

Example 2: W25Q128JW Datasheet information is given in the figure below.

3. At least one byte of data input is required for Page Program, Quad Page Program and Program Security Registers, up to 256 bytes of data input. If more than 256 bytes of data are sent to the device, the addressing will wrap to the beginning of the page and overwrite previously sent data.
4. Write Status Register-1 (01h) can also be used to program Status Register-1&2, see section 8.2.5.
5. Security Register Address:

Security Register 1:	A23-16 = 00h;	A15-8 = 10h;	A7-0 = byte address
Security Register 2:	A23-16 = 00h;	A15-8 = 20h;	A7-0 = byte address
Security Register 3:	A23-16 = 00h;	A15-8 = 30h;	A7-0 = byte address

Figure 6: W25Q128JW Datasheet Information

According to the Datasheet:
sreg_block_size: 256. Security register size. Unit: byte.
sreg_min_num: 1. Minimum security register number.
sreg_max_num: 3. Maximum security register number.

2.2.1.3. halSpiFlashType_t

The enumeration of NOR flash manufacturer types:

```
typedef enum
{
    HAL_SPI_FLASH_TYPE_UNKNOWN,
    HAL_SPI_FLASH_TYPE_GD,
    HAL_SPI_FLASH_TYPE_WINBOND,
    HAL_SPI_FLASH_TYPE_XTX,
    HAL_SPI_FLASH_TYPE_XMCA,
```

```
    HAL_SPI_FLASH_TYPE_XMCC,
    HAL_SPI_FLASH_TYPE_XMCB,
    HAL_SPI_FLASH_TYPE_PUYA,
} halSpiFlashType_t
```

- **Member**

Member	Description
HAL_SPI_FLASH_TYPE_UNKNOWN	Unknown type
HAL_SPI_FLASH_TYPE_GD	GigaDevice
HAL_SPI_FLASH_TYPE_WINBOND	WINBOND
HAL_SPI_FLASH_TYPE_XTX	XTX
HAL_SPI_FLASH_TYPE_XMCA	XMC-A
HAL_SPI_FLASH_TYPE_XMCC	XMC-C
HAL_SPI_FLASH_TYPE_XMCB	XMC-B
HAL_SPI_FLASH_TYPE_PUYA	PUYA

NOTE

If the selected NOR flash type is not included in the above manufacturer types, you can try to select a similar type by referring to the Datasheet. But you need to verify whether the selected flash can be successfully driven. If the flash cannot be driven successfully, please contact Quectel Technical Support.

2.2.1.4. halSpiFlashUidType_t

The enumeration of UID types:

```
typedef enum
{
    HAL_SPI_FLASH_UID_NONE,
    HAL_SPI_FLASH_UID_4BH_8,
    HAL_SPI_FLASH_UID_4BH_16,
    HAL_SPI_FLASH_UID_SFDP_80H_12,
    HAL_SPI_FLASH_UID_SFDP_94H_16,
    HAL_SPI_FLASH_UID_SFDP_194H_16,
} halSpiFlashUidType_t
```

- **Member**

Member	Description
HAL_SPI_FLASH_UID_NONE	None.
HAL_SPI_FLASH_UID_4BH_8	The command for reading 8-bit UID is **4BH**.
HAL_SPI_FLASH_UID_4BH_16	The command to read 16-bit UID is **4BH**.
HAL_SPI_FLASH_UID_SFDP_80H_12	UID is at offset 80H of SFDP Table and the length is 12 bits.
HAL_SPI_FLASH_UID_SFDP_94H_16	UID is at offset 94H of SFDP Table and the length is 16 bits.
HAL_SPI_FLASH_UID_SFDP_194H_16	UID is at offset 194H of SFDP Table and the length is 16 bits.

2.2.1.5. halSpiFlashCpidType_t

The enumeration of CPID types:

```
typedef enum
{
    HAL_SPI_FLASH_CPID_NONE,
    HAL_SPI_FLASH_CPID_4BH, // byte [17:16]
} halSpiFlashCpidType_t
```

- **Member**

Member	Description
HAL_SPI_FLASH_CPID_NONE	None.
HAL_SPI_FLASH_CPID_4BH	Command for reading CPID is **4BH**.

2.2.1.6. halSpiFlashWpType_t

The enumeration of write protection types:

```
typedef enum
{
    HAL_SPI_FLASH_WP_NONE,
    HAL_SPI_FLASH_WP_GD,
    HAL_SPI_FLASH_WP_XMCA,
```

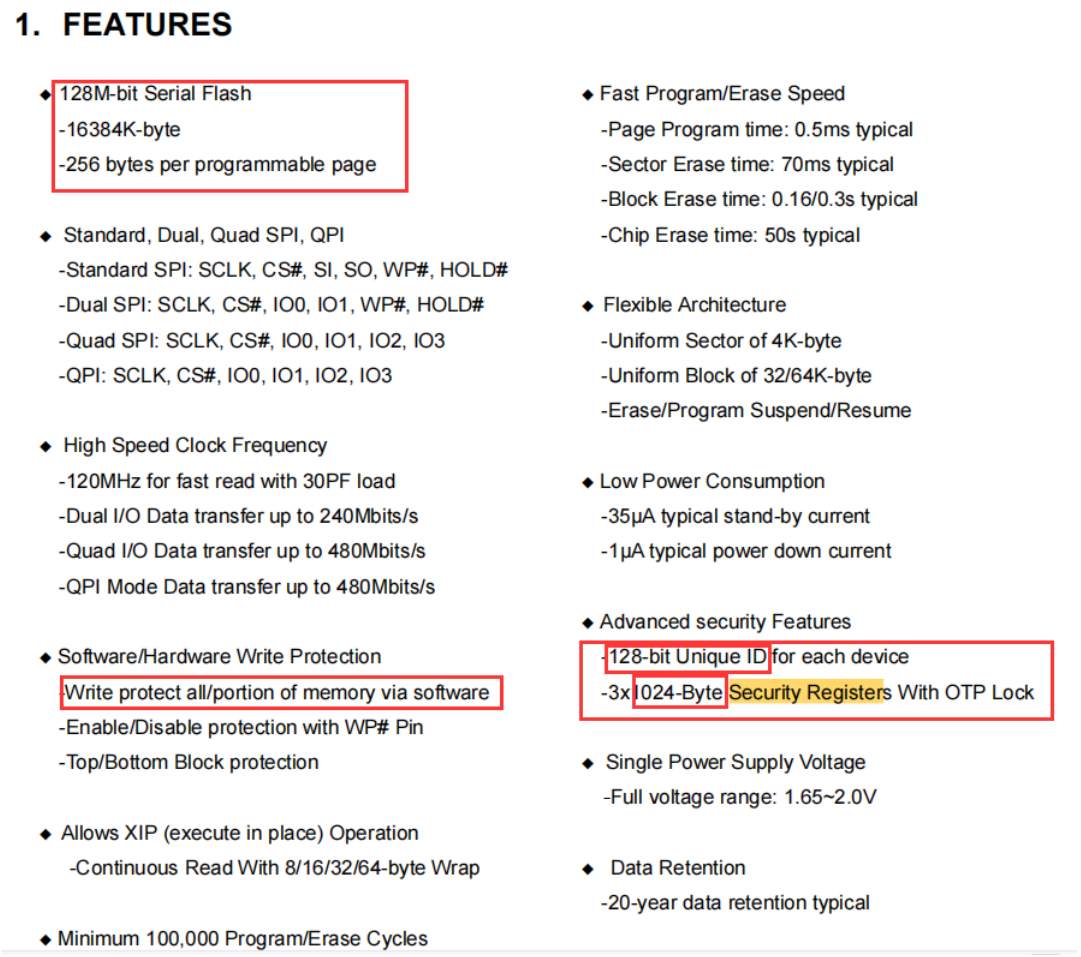
} halSpiFlashWpType_t

- **Member**

Member	Description
HAL_SPI_FLASH_WP_NONE	None.
HAL_SPI_FLASH_WP_GD	Compatibility mode of write protection is BP0/1/2/3/4/CMP.
HAL_SPI_FLASH_WP_XMCA	Compatibility mode of write protection is SR2/3/4/5.

2.2.1.7. Other Feature Parameters

GD25LQ128D is used in the examples analyzing the parameters in *hal_spi_flash_prop.csv*. GD25LQ128D Datasheet information, including features, status register and supported commands, is given in the figure below.



1. FEATURES

- 128M-bit Serial Flash
  - -16384K-byte
  - -256 bytes per programmable page
- Standard, Dual, Quad SPI, QPI
  - -Standard SPI: SCLK, CS#, SI, SO, WP#, HOLD#
  - -Dual SPI: SCLK, CS#, IO0, IO1, WP#, HOLD#
  - -Quad SPI: SCLK, CS#, IO0, IO1, IO2, IO3
  - -QPI: SCLK, CS#, IO0, IO1, IO2, IO3
- High Speed Clock Frequency
  - -120MHz for fast read with 30PF load
  - -Dual I/O Data transfer up to 240Mbits/s
  - -Quad I/O Data transfer up to 480Mbits/s
  - -QPI Mode Data transfer up to 480Mbits/s
- Software/Hardware Write Protection
  - -Write protect all/portion of memory via software
  - -Enable/Disable protection with WP# Pin
  - -Top/Bottom Block protection
- Allows XIP (execute in place) Operation
  - -Continuous Read With 8/16/32/64-byte Wrap
- Minimum 100,000 Program/Erase Cycles
- Fast Program/Erase Speed
  - -Page Program time: 0.5ms typical
  - -Sector Erase time: 70ms typical
  - -Block Erase time: 0.16/0.3s typical
  - -Chip Erase time: 50s typical
- Flexible Architecture
  - -Uniform Sector of 4K-byte
  - -Uniform Block of 32/64K-byte
  - -Erase/Program Suspend/Resume
- Low Power Consumption
  - -35μA typical stand-by current
  - -1μA typical power down current
- Advanced security Features
  - -128-bit Unique ID for each device
  - -3x1024-Byte Security Registers With OTP Lock
- Single Power Supply Voltage
  - -Full voltage range: 1.65~2.0V
- Data Retention
  - -20-year data retention typical

Figure 7: Feature Introduction

S15	S14	S13	S12	S11	S10	S9	S8
SUS1	CMP	LB3	LB2	LB1	SUS2	QE	SRP1

S7	S6	S5	S4	S3	S2	S1	S0
SRP0	BP4	BP3	BP2	BP1	BP0	WEL	WIP

Figure 8: Status Register Introduction

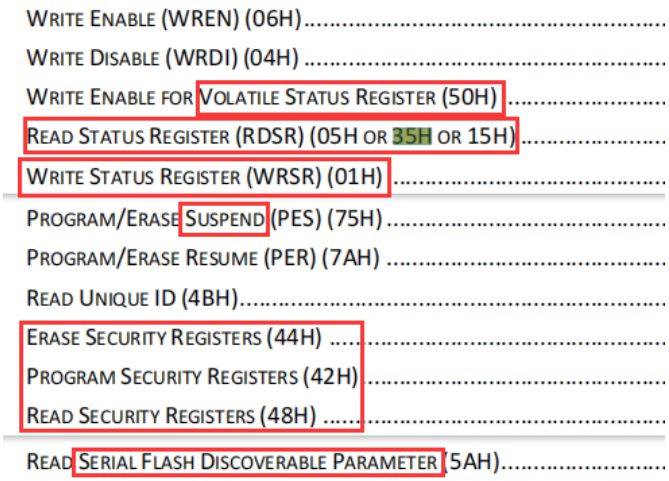


Figure 9: Supported Commands

According to the Datasheet information marked in red:

capacity: 0x1000000. It indicates GD25LQ128C with 128 Mbit capacity.
volatile_sr_en: 1. Support volatile status register.
suspend_en: 1. Support the suspend and resume of read-write operation.
sfdp_en: 1. Generally supported.
write_sr12: 1. Support writing data to SR1 and SR2 status registers through **01H**.
has_sr2: 1. Support reading data from SR2 through **35H**.
has_sus1: 1. SUS1 exists in S15 of the status register.
has_sus2: 1. SUS2 exists in S10 of the status register.

3 6-wire SPI NOR Flash File System Mounting API

3.1. Header File

ql_api_spi6_ext_nor_flash.h, the header file of 6-wire SPI NOR flash file system mounting API, is in the *\components\ql-kernel\inc* directory. Unless otherwise specified, all header files mentioned in this document are in this directory.

3.2. API Overview

Table 2: API Overview

Function	Description
ql_spi6_ext_nor_flash_init()	Initializes the dedicated 6-wire SPI pins.
ql_spi6_ext_nor_flash_deinit()	De-initializes the dedicated 6-wire SPI pins.
ql_spi6_ext_nor_flash_sffs_mkfs()	Formats 6-wire SPI NOR flash to the SFFS format.
ql_spi6_ext_nor_flash_sffs_mount()	Mounts 6-wire SPI NOR flash to the SFFS file system.
ql_spi6_ext_nor_flash_sffs_unmount()	Unmounts 6-wire SPI NOR flash from the SFFS file system.

NOTE

In CSDK package, the macro definition **EXT_DEFAULT_RESERVED_SPACE_SIZE** in *ql_hal_spi_flash_prop.c* in the *\components\hal\src* directory sets reserved NOR flash space by default. This reserved space is not used for mounting the file system. You can modify it as needed.
For compatibility purposes:

- EC600U series module reserves the first 0x80000 bytes of NOR flash for not mounting the file

system;

```
#define EXT_DEFAULT_RESERVED_SPACE_SIZE     0x80000
```

- ECx00G family and EG800G series modules reserve the first 0x0 bytes of NOR flash for not mounting the file system.

```
#define EXT_DEFAULT_RESERVED_SPACE_SIZE     0x0
```

3.3. API Description

3.3.1. ql_spi6_ext_nor_flash_init

This function initializes the dedicated 6-wire SPI pins and configures them to 6-wire SPI.

- **Prototype**

```
void ql_spi6_ext_nor_flash_init(void)
```

- **Parameter**

None

- **Return Value**

None

3.3.2. ql_spi6_ext_nor_flash_deinit

This function de-initializes the dedicated 6-wire SPI pins and restores them to the default feature.

- **Prototype**

```
void ql_spi6_ext_nor_flash_deinit(void)
```

- **Parameter**

None

- **Return Value**

None

3.3.3. ql_spi6_ext_nor_flash_sffs_mkfs

This function formats 6-wire SPI NOR flash to the SFFS format. After this function is executed successfully and you read the mounted NOR flash again through the file system, the result will be empty.

- **Prototype**

```
ql_errcode_spi6_nor_e ql_spi6_ext_nor_flash_sffs_mkfs(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.3.3.1*** for details.

3.3.3.1. ql_errcode_spi6_nor_e

The result codes of 6-wire SPI NOR flash file system mounting API indicate if the function is executed successfully or not. If function execution fails, error cause is returned. The enumeration of result codes:

```
typedef enum
{
    QL_SPI6_EXT_NOR_FLASH_SUCCESS,
    QL_SPI6_EXT_NOR_FLASH_NOT_INIT_ERROR,
    QL_SPI6_EXT_NOR_FLASH_CREATE_FBDEV2_ERROR,
QL_SPI6_EXT_NOR_FLASH_REPEAT_MOUNT_ERROR,
QL_SPI6_EXT_NOR_FLASH_FIND_BLOCK_DEVICE_ERROR,
    QL_SPI6_EXT_NOR_FLASH_SFFS_MOUNT_ERROR,
    QL_SPI6_EXT_NOR_FLASH_SFFS_UNMOUNT_ERROR,
    QL_SPI6_EXT_NOR_FLASH_PARTITION_INFO_ERROR
}ql_errcode_spi6_nor_e
```

- **Member**

Member	Description
QL_SPI6_EXT_NOR_FLASH_SUCCESS	Successful execution.
QL_SPI6_EXT_NOR_FLASH_NOT_INIT_ERROR	Failure to initialize the SPI NOR flash.
QL_SPI6_EXT_NOR_FLASH_CREATE_FBDEV2_ERROR	Failure to create a block device.

QL_SPI6_EXT_NOR_FLASH_REPEAT_MOUNT_ERROR	Failure to mount repeatedly.
QL_SPI6_EXT_NOR_FLASH_FIND_BLOCK_DEVICE_ERROR	Failure to find a block device.
QL_SPI6_EXT_NOR_FLASH_SFFS_MOUNT_ERROR	Failure to mount file system.
QL_SPI6_EXT_NOR_FLASH_SFFS_UNMOUNT_ERROR	Failure to unmount file system.
QL_SPI6_EXT_NOR_FLASH_PARTITION_INFO_ERROR	Partition information error.

3.3.4. ql_spi6_ext_nor_flash_sffs_mount

This function mounts 6-wire SPI NOR flash to the SFFS file system. After the mounting is successful, the file system API can be used to perform the read-write operation.

- **Prototype**

```
ql_errcode_spi6_nor_e ql_spi6_ext_nor_flash_sffs_mount(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.3.3.1*** for details.

3.3.5. ql_spi6_ext_nor_flash_sffs_unmount

This function unmounts 6-wire SPI NOR flash from the SFFS file system.

- **Prototype**

```
ql_errcode_spi6_nor_e ql_spi6_ext_nor_flash_sffs_unmount(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.3.3.1*** for details.

4 Example and Debugging

This chapter outlines how to use the above API for development and simple debugging of 6-wire SPI NOR flash file system mounting on application layer.

4.1. Development Example

4.1.1. Example Description

spi6_ext_nor_flash_demo.c, the example file of the above API is provided in the CSDK of the module and it is in the *\components\ql-application\spi6_ext_nor_flash* directory. Before development, you need to ensure that the TE-A in use is soldered with NOR flash and that it is consistent with the 6-wire SPI interface in use.

Initialize the dedicated 6-wire SPI pins, mount it to the file system and call the file system API operate the mounted NOR flash. See ***document [2]*** for details.

```
QlOSStatus ql_spi6_ext_nor_flash_demo_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;

    //QL_SPI6_EXT_NOR_LOG("enter ql_spi6_ext__nor_flash_demo_init");
    err = ql_rtos_task_create(&spi6_ext_nor_flash_demo_task, SPI6_EXT_NOR_FLASH_DEMO_TASK_STACK_SIZE, SPI6_EXT_NOR_FLASH_DEMO_TASK_PRIO, "ql_spi6_nor",
    if(err != QL_OSI_SUCCESS)
    {
        QL_SPI6_EXT_NOR_LOG("demo_task created failed");
        return err;
    }

    return err;
}
```

Figure 10: Example Initialization

```
static void ql_spi6_ext_nor_flash_demo_task_pthread(void *ctx)
{
    QlOSStatus erro = 0;
    ql_errcode_spi6_nor_e ret;

    ql_spi6_ext_nor_flash_init();
    ret = ql_spi6_ext_nor_flash_sffs_mount();
    if(QL_SPI6_EXT_NOR_FLASH_SUCCESS != ret)
    {
#ifdef QL_APP_FEATURE_FILE
      int64 free_size = ql_fs_free_size("EFS:");
      QL_SPI6_EXT_NOR_LOG("fs free_size :%d", free_size);
#endif
      goto QL_SPI6_TASK_EXIT;
    }
#ifdef QL_APP_FEATURE_FILE
    int fd = 0;
    int64 err = 0;
    char buffer[100];
    char *str = QL_EFS_TEST_STR;

    int64 free_size = ql_fs_free_size("EFS:");
    QL_SPI6_EXT_NOR_LOG("fs free_size :%d", free_size);

    fd = ql_fopen(QL_EFS_FILE_PATH, "wb+");
    if(fd < 0)
    {
      QL_SPI6_EXT_NOR_LOG("open file failed");
      err = fd;
      goto exit;
    }

    err = ql_fwrite(str, strlen(str) + 1, 1, fd);   //strlen not include '\0'
    if(err < 0)
    {
      QL_SPI6_EXT_NOR_LOG("write file failed");
      ql_fclose(fd);
      goto exit;
    }

    err = ql_frewind(fd);
    if(err < 0)
    {
      QL_SPI6_EXT_NOR_LOG("rewind file failed");
      ql_fclose(fd);
      goto exit;
    }

    err = ql_fread(buffer, ql_fsize(fd), 1, fd);
```

Figure 11: Initialize the Pins and Mount the Device

4.1.2. Example Auto-Start

The example above is disabled by default in *ql_init_demo_thread()*. Uncomment the codes before testing, as shown in the figure below:

```
static void ql_init_demo_thread(void *param)
{
    QL_INIT_LOG("init demo thread enter, param 0x%x", param);

/*Caution:If the macro of secure boot and the function are opened, download firmware and restart will enable secure boot.
          the secret key cannot be changed forever*/
#ifdef QL_APP_FEATURE_SECURE_BOOT
    //ql_dev_enable_secure_boot();
#endif
#ifdef QL_APP_FEATURE_SPI_NOR_FLASH
    //ql_spi_nor_flash_demo_init();
#endif
#ifdef QL_APP_FEATURE_SPI6_EXT_NOR
    ql_spi6_ext_nor_flash_demo_init();
#endif
#ifdef QL_APP_FEATURE_SPI_NAND_FLASH
    //ql_spi_nand_flash_demo_init();
#endif
```

Figure 12: *ql_init_demo_thread()*

4.2. Feature Debugging

4.2.1. Debugging Preparation

6-wire SPI NOR flash file system mounting on the module can be debugged through Quectel LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to the PC with a USB cable. The USB AP Log Port and USB DIAG Port mainly displays the system debugging information. You can view information of the example through USB AP Log Port or USB DIAG Port after capturing the log with cooltools. For details about log capture, see ***documents [3]*** and ***[4]***.

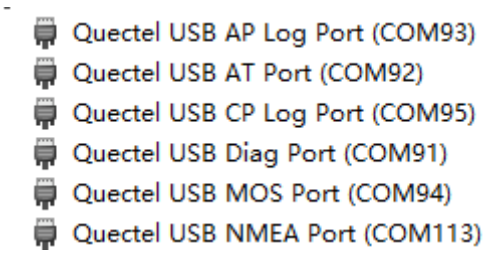


Figure 13: Ports in Device Manager (EC600U Series)

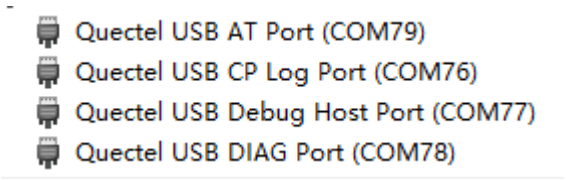


Figure 14: Ports in Device Manager (ECx00G Family/EG800G Series)

NOTE

For EC600U series module, cooltools is used to capture logs through USB AP Log Port; see ***document [3]*** for details on log capture. For ECx00G family and EG800G series modules, ArmLogel is used to capture logs through USB DIAG Port; see ***document [4]*** for details on log capture.

4.2.2. Feature Debugging

ql_spi6_ext_nor_flash_app_init() runs automatically after the module is booted. You can view the mounting information of 6-wire SPI NOR flash file system through the log. The debugging information of AP log port is shown in the figure below:

```
QUEV/I  [FS M][quec_set_spi6_ext_nor_flash_capacity, 156] ext flash capacity:0xf80000
QOPN/I  [ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 63] fs free_size :15797892
QOPN/I  [ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 97] file read result is 1234567890abcdefg
QOPN/I  [ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 108] open file failed,errcode is 83030260
QOPN/I  [ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 112] sffs mount ret 0
QOPN/I  [ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 127] close file
QOPN/I  [ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 147] ql_rtos_task_delete
```

Figure 15: Log Information

If the module fails to initialize the SPI NOR flash, the general failure causes are:

1. **MID cannot be read or the module does not support the flash type.** The log information of the failed initialization is shown in the figure below:

```
    /I : Not support nor mid:0x0
FSYS/I : failed to open spi flash SFL2
QUEV/I : [FS M][quec_spi6_ext_nor_flash_mount, 185] failed to find block device FEXT
QOPN/I : [ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 52] fs free_size :-2096954973
QOPN/I : [ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 147] ql_rtos_task_delete
```

Figure 16: Logs Without Reading MID

As shown in the figure above, if mid is 0x0, it indicates that the actual mid value cannot be read. And the possible causes are:

- No SPI NOR flash is connected to TE-A.
- Dedicated 6-wire SPI pin is not initialized successfully. And the possible causes are:
  1) Initialization function is not called.
  2) Parameter configuration of *quec_boot_spi6_pin_cfg_map[]* in *quec_boot_pin_map.c* is incorrect.
  3) Configured 6-wire SPI interface is inconsistent with the actual one.

16:32:49.011	5400	/I	Not support nor mid:0x218c4

Figure 17: Failure to Find Type Information

As shown in the figure above, if mid is not 0, it indicates that the type information cannot be found in *hal_spi_flash_prop.csv*, and you can add the corresponding information to *hal_spi_flash_prop.csv*. See ***Chapter 2*** for details.

2. **Failed to mount the NOR flash.** The log information of the failed initialization is shown in the figure below:

FSYS/I	FBDEV: scan EB quickinit/0 invalid/3968 samelb/0 erase count/15728640/0 next use/0
FSYS/I	SFFS mount, device block size/500 block count/31735 cache count 4
FSYS/E	SFFS mount wrong root tag
FSYS/E	failed to mount /ext
QOPN/I	[ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 52] fs free_size :1242392
QOPN/I	[ql_SPI6_NOR_DEMO][ql_spi6_ext_nor_flash_demo_task_pthread, 147] ql_rtos_task_delete
QOPN/I	[ql_OSI][ql_rtos_task_delete, 192] remove task 80BFC4E0 ok

Figure 18: Log of Failed Mounting

If NOR flash mounting fails, you need to format it first, and then mount it again. The solution is shown as below:

- Call *ql_spi6_ext_nor_flash_sffs_mkfs()* to format the NOR flash to be mounted.
- Use **AT+QEFSMKFS="SFFS"** to format the NOR flash to be mounted.

5 Appendix References

Table 3: Related Documents

Document Name	
[1]	Quectel_ECx00G&EC600U&EG800G_Series_QuecOpen(SDK)_Quick_Start_Guide
[2]	Quectel_EC200D&ECx00G&EC600U&EG800G_Series_QuecOpen(SDK)_File_System_API_Reference_Manual
[3]	Quectel_EC600U_Series_QuecOpen(SDK)_Log_Capture_Guide
[4]	Quectel_ECx00G&EG800G_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
App	Application
API	Application Programming Interface
CPID	Consumer Product Identification
CSDK	Customized Software Development Kit
EVB	Evaluation Board
JEDEC	Joint Electron Device Engineering Council
MID	Manufacturer Identification
PC	Personal Computer
RTOS	Real-time Operating System
SFDP	Serial Flash Discoverable Parameters
SPI	Serial Peripheral Interface

UID	Unique Identification
USB	Universal Serial Bus